

{creative} [coding]

Dokumentation Woche 2

{Montag} 04.05.2026

Vormittag

- Allgemeine Vorstellung von Ted Davis und seiner Arbeit
- Vorstellung von verschiedenen Tools: P5Live, p5.js, Hydra

Nachmittag

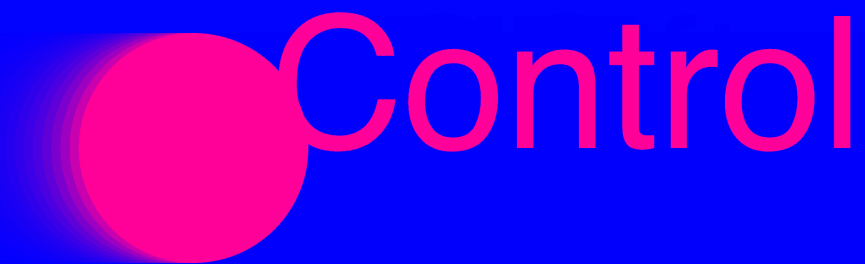
- Audio-Reaktive Live-Coding Experimente
- Kennenlernen von Snippets
- P5.js Library kennenlernen und integrieren
- Vorstellung der Plattform Studel

☆ BASIC EXPERIMENTE P5LIVE 1

```
function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  // transparenter HG > Spuren
  background(0, 0, 255, 55)
  // zeichnet Kreis, von rechts nach links
  // startet immer wieder links
  circle(frameCount*10%width, height/2, 250)
  noStroke()
  print(mouseX)
  fill(255, 0, 153)

  // Text Styling
  textSize(200)
  textAlign(CENTER) //bla bla bla
  // zeichnet zuletzt gedrückte Taste
  text(key, width/2, height/2)
}
```



☆ BASIC EXPERIMENTE P5LIVE 2

```
function setup() {  
  createCanvas(windowWidth, windowHeight)  
}  
  
function draw() {  
  // transparenter HG > Spuren  
  background(0, 15, 255, 15)  
  // ändert die Farbe mit der Zeit:  
  // Rot variiert (0–255), Grün 100% an, Blau 0%  
  // erzeugt fließenden Übergang zwischen Grün und Gelb  
  fill(frameCount % 255, 255, 0)  
  // zeichnet Kreis an Maus-Pos, ändert Grösse  
  circle(mouseX, mouseY, frameCount % 200)  
  
  // gibt aktuelle Frame-Nummer aus > Debugging  
  print(frameCount)  
}
```



☆ AUDIOREACTIVE

```
function setup() {
  createCanvas(windowWidth, windowHeight)

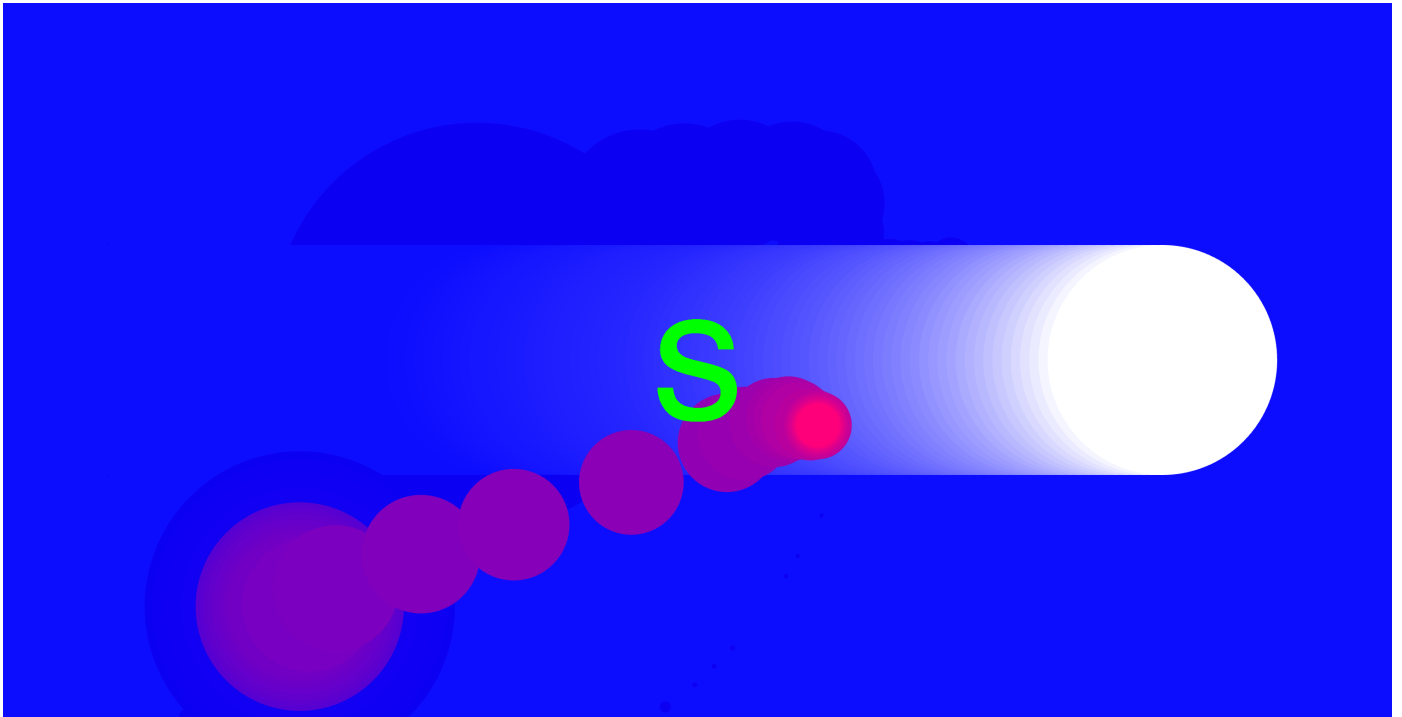
  setupAudio(true) // global vars
  // a5.ease = .075 // set easing
}

function draw() {
  /* audio vars: amp, ampEase, fft, fftEase, waveform, waveformEase */
  updateAudio()

  // Kreis von links nach rechts
  background(0, 0, 255, 10)
  fill(255)
  circle(frameCount * 10 % width, height / 2, 250)
  noStroke()
  print(mouseX)

  // Kreis der mit der Maus hinterhergeht
  // ändert sich auf Basis der Lautstärke des Mikrofons
  fill (255, 0, 123)
  circle(mouseX, mouseY, ampEase*10)

  // text that appears whenever you press a key
  push()
  fill(0, 255, 0)
  textSize(200)
  textAlign(CENTER, CENTER)
  text(key, width / 2, height / 2)
  pop()
}
```



☆ CAMERA SNIPPET

```
let capture, scl = 1, // Skalierungsfaktor Bild
    rot = 0

function setup() {
  createCanvas(windowWidth, windowHeight)

  // Aktiviert Webcam
  capture = createCapture(VIDEO)
  // Auflösung
  capture.size(200, 180)
  // versteckt originales html-videoelement
  capture.hide()
  // setzt Ursprung in Mitte
  imageMode(CENTER)
}

// gibt Kamera-Bild aus
function draw() {
  image(capture, mouseX, mouseY, capture.width * scl, capture.height * scl)
}
```




☆ P5LIVE LIBRARY MBWalker

```
// externe Bibliothek wird geladen
let libs = ['https://tetunori.github.io/BMWalker.js/dist/v0.6.1/bmwwalker.js']

const bmw = new BMWWalker();

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  background(115, 0, 73)
  //clear()
  // Grösse der Figur
  const walkerHeight = 200;
  // holt die aktuellen Punkte des Walkers
  // > Körperpunkte / Gelenke
  const markers = bmw.getMarkers(walkerHeight);

  // Alternative: Walker läuft mit Maus
  //translate(mouseX, mouseY)
  // verschiebt Pos von Ursprung in Mitte
  translate(width / 2, height / 2)
  // zeichnet um jeden Marker einen Kreis
  markers.forEach((m) => {circle(m.x, m.y, 30);});
}
```



{Dienstag} 05.05.2026

Vormittag

- Allgemeine Vorstellung von Alper Yagcioglu und seiner Arbeit
- Generierung einer «Maschine» in P5Live

Nachmittag

- verschiedene Experimente in P5Live

★ MASCHINE

```
let slider, checkbox, button, colorPicker, dropdown, input, sliderText, buttonText, radioButton;
let bgColor;
let colorText;
let positionDOM;

function setup() {
  createCanvas(windowWidth, windowHeight);
  positionDOM = width - 400

  // ----- Form -----
  //checkbox (Erstellung Checkbox / Position)
  checkbox = createCheckbox('Show Form', true);
  checkbox.position(width - 400, 20);
```

```

// Form Breite verändern
slider = createSlider(50, height - 100, 200);
slider.position(positionDOM, 60);

// Form Höhe verändern
slider2 = createSlider(50, height - 100, 200);
slider2.position(positionDOM, 80);

// Color Picker Kreis
colorPicker = createColorPicker('#ff0000');
colorPicker.position(positionDOM, 120);

// Dropdown Form
dropdown = createSelect();
dropdown.position(positionDOM, 160);
dropdown.option('Circle');
dropdown.option('Square');
dropdown.option('Circle2');
rectMode(CENTER);

// ----- Hintergrund -----
// Button Zufälliger HG
button = createButton('Random Background');
button.position(positionDOM, 210);
button.mousePressed(() => {
  bgColor = color(random(255), random(255), random(255));
});
bgColor = color(220);

// ----- Text -----
// Input field
input = createInput('Type text');
input.position(positionDOM, 260);
textAlign(CENTER, CENTER);

// Textgröße
sliderText = createSlider(50, height - 100, 200);
sliderText.position(positionDOM, 290);

// Dropdown Font
dropdownFont = createSelect();
dropdownFont.position(positionDOM, 320);
dropdownFont.option('Helvetica');
dropdownFont.option('Courier');
dropdownFont.option('Verdana');
dropdownFont.option('Times New Roman');
rectMode(CENTER);

// Button zufällige Textfarbe
buttonText = createButton('Random Color Text');
buttonText.position(positionDOM, 350);
buttonText.mousePressed(() => {
  colorText = color(random(255), random(255), random(255));
});
colorText = color(220);

// ----- Kontur -----

```

```

// Konturdicke verändern
sliderStroke = createSlider(2, 50, 5);
sliderStroke.position(positionDOM, 450);

//Radio button
radio = createRadio();
radio.option('Black');
radio.option('White');
radio.option('None');
radio.selected('None');
radio.position(positionDOM, 420);
}

// Zeichnet Kreis / Rechteck
function draw() {
  background(bgColor);
  fill(colorPicker.value());
  strokeWeight(sliderStroke.value());

  if (radio.value() === 'Black') stroke(0);
  if (radio.value() === 'White') stroke(255);
  if (radio.value() === 'None') noStroke();

  //if-loop - Dropdown - Form auswählen
  if (checkbox.checked()) {
    if (dropdown.value() === 'Circle') {
      ellipse(width / 2, height / 2, slider.value(), slider2.value());
    }
    else if (dropdown.value() === 'Square') {
      rect(width / 2, height / 2, slider.value(), slider2.value());
    }
    else if (dropdown.value() === 'Circle2') {
      ellipse(width / 2 + 100, height / 2, slider.value(), slider2.value());
      ellipse(width / 2 - 100, height / 2, slider.value(), slider2.value());
    }
  }

  // if-loop - Font auswählen
  if(dropdownFont.value() === 'Helvetica') {
    textFont('Helvetica')
  }
  else if(dropdownFont.value() === 'Courier') {
    textFont('Courier')
  }
  else if(dropdownFont.value() === 'Verdana') {
    textFont('Verdana')
  }
  else if(dropdownFont.value() === 'Times New Roman') {
    textFont('Times New Roman')
  }

  // Gibt Text von Input aus
  // zufällige Text-Farbe
  push()
  noStroke()
  fill(colorText)
  textSize(sliderText.value());

```

```
text(input.value(), width / 2, height / 2);  
pop()  
}
```



★ KREIS MIT DYNAMISCHER PINSELPITZE

```
function setup() {  
  createCanvas(windowWidth, windowHeight);  
  background(7, 29, 232);  
  // Anfangsstrichfarbe  
  stroke(255, 76, 5);  
}
```

```

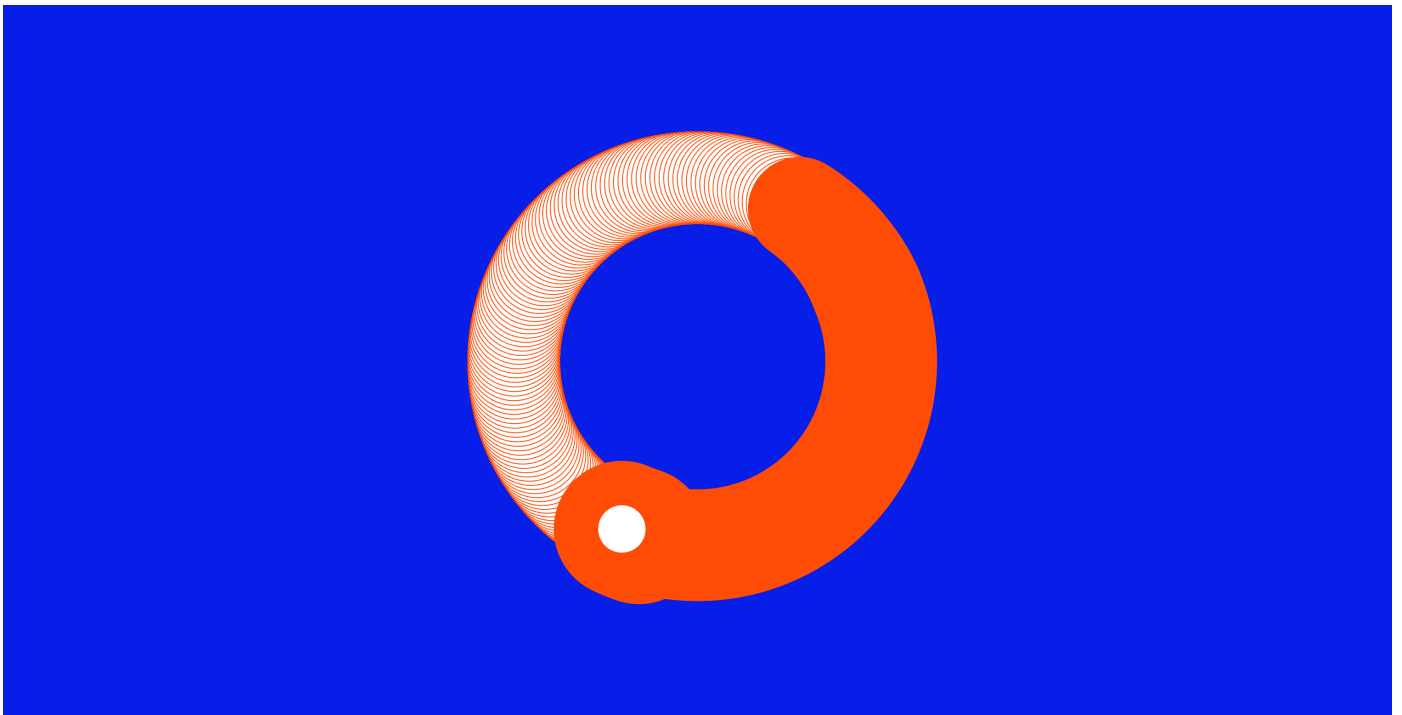
}
function draw() {
  // Dynamische Stiftgröße:
  // Sinus > zwischen -1 und 1
  // map > übersetzt in Werte zwischen 10 und 50
  let pen1 = map(sin(frameCount * 0.025), -1, 1, 10, 50)

  // Mapping der Maus-X-Pos auf halbe Breite
  let differentX = map(mouseX, 0, width, 0, width / 2)
  // zeichnet Ellipse, bewegt sich kreisförmig in Mitte
  ellipse(width/2 + cos(frameCount*0.025) * 200, height/2 + sin(frameCount * 0.025) * 200, 100)

  // Wenn Maus gedrückt wird:
  // gibt es Strichfarbe & dynamische Breite
  if (mouseIsPressed == true) {
    stroke(255, 76, 5)
    strokeWeight(pen1)
  }
  // Speichert aktuelle Maus-Pos
  prevX = mouseX;
  prevY = mouseY;
}

// Bild wird gespeichert, wenn Shift + s geklickt wird
function keyPressed(){
  if (key == 'S') {
    save('drawing.png')
  }
}
}

```



★ STIFT MIT DYNAMISCHER PINSELSPITZE

```

function setup() {
  createCanvas(windowWidth, windowHeight);
}

```

```

background(200);
stroke(0);

}
function draw() {
  // Dynamische Stiftgrösse:
  // Sinus = pulsierende Veränderung > Ergebnis Wert zwischen 40 – 60
  let pen1 = map(sin(frameCount * 0.025), -1, 1, 40, 60)

  // Wenn die Maus geklickt wird:
  if (mouseIsPressed == true) {
    // 1. Ellipse schwarz
    stroke(0)
    strokeWeight(pen1)
    ellipse(mouseX, mouseY, pen1, pen1);

    // 1. Ellipse weiss
    stroke(255)
    ellipse(mouseX+20, mouseY+30, pen1, pen1);
  }
}

// Wenn Shift + s auf Tastatur gedrückt wird > Screenshot
function keyPressed(){
  if(key == 'S') {
    save('drawing.png')
  }
}

```



★ PINSEL ALS SMILEY

```

function setup() {
  createCanvas(windowWidth, windowHeight);
  background(200);

```

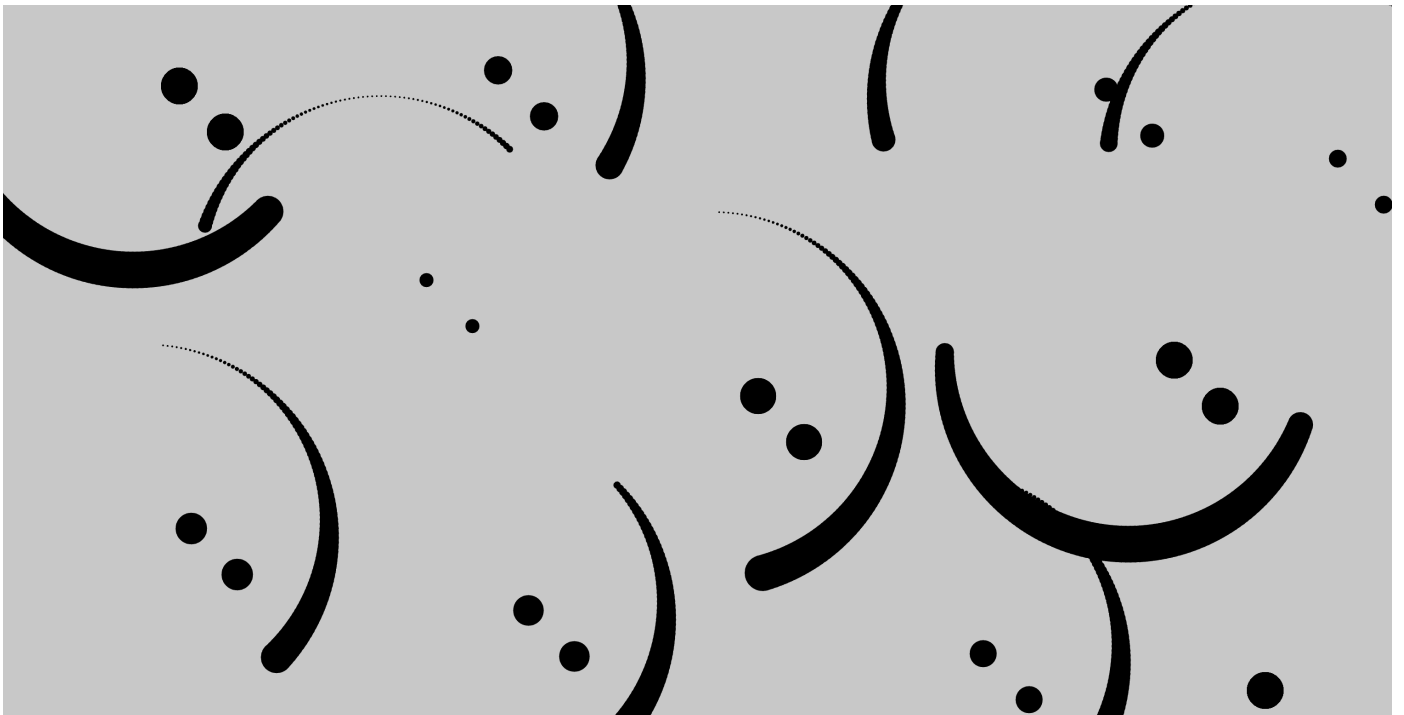
```

stroke(0);
}
function draw() {
  // Dynamische Stiftgrösse:
  // Sinus > zwischen -1 bis 1
  // map > übersetzt in Werte zwischen 1 und 20
  let pen1 = map(sin(frameCount * 0.025), -1, 1, 1, 20)

  // nur zeichnen, wenn Maus geklickt wird:
  if (mouseIsPressed == true) {
    stroke(0)
    strokeWeight(pen1)
    // 1. Ellipse bei Maus
    ellipse(mouseX, mouseY, pen1, pen1);
    // 2. Ellipse leicht versetzt zur Maus
    ellipse(mouseX+50, mouseY+50, pen1);
    // 3. Ellipsen versetzt zur Maus &
    ellipse(mouseX-50+ cos(frameCount*0.025) * 200, mouseY + sin (frameCount*0.025) *200, pen1,
pen1);
  }
}

// Wenn Shift + s auf Tastatur gedrückt wird > Screenshot
function keyPressed(){
  if(key == 'S') {
    save('drawing.png')
  }
}

```



★ DYNAMISCHE PINSELGRÖSSE

```

function setup() {
  createCanvas(windowWidth, windowHeight);
  background(238, 255, 184);
}

```



```

stroke(0);
}

function draw() {
  // Dynamische Stiftgrösse:
  // Sinus > pulsierende Veränderung
  // map > übersetzt in Werte zwischen -1, 1 und 1, 30
  let pen1 = map(sin(frameCount*0.03),-1,1,1,30)

  // zeichnet, wenn Maus geklickt
  if (mouseIsPressed) {
    stroke(0);
    strokeWeight(pen1);
    line(prevX, prevY, mouseX, mouseY);
    stroke(255)
  }
  // Linie der vorherigen Maus-Pos zur aktuellen
  prevX = mouseX;
  prevY = mouseY;
}

// Wenn Shift + s auf Tastatur gedrückt wird > Screenshot
function keyPressed(){
  if(key == 'S'){
    save('drawing.png')
  }
}

```



★ BEISPIEL VERHALTEN SINUS & COSINUS

```

let number = 5
let speedX
let speedY

```

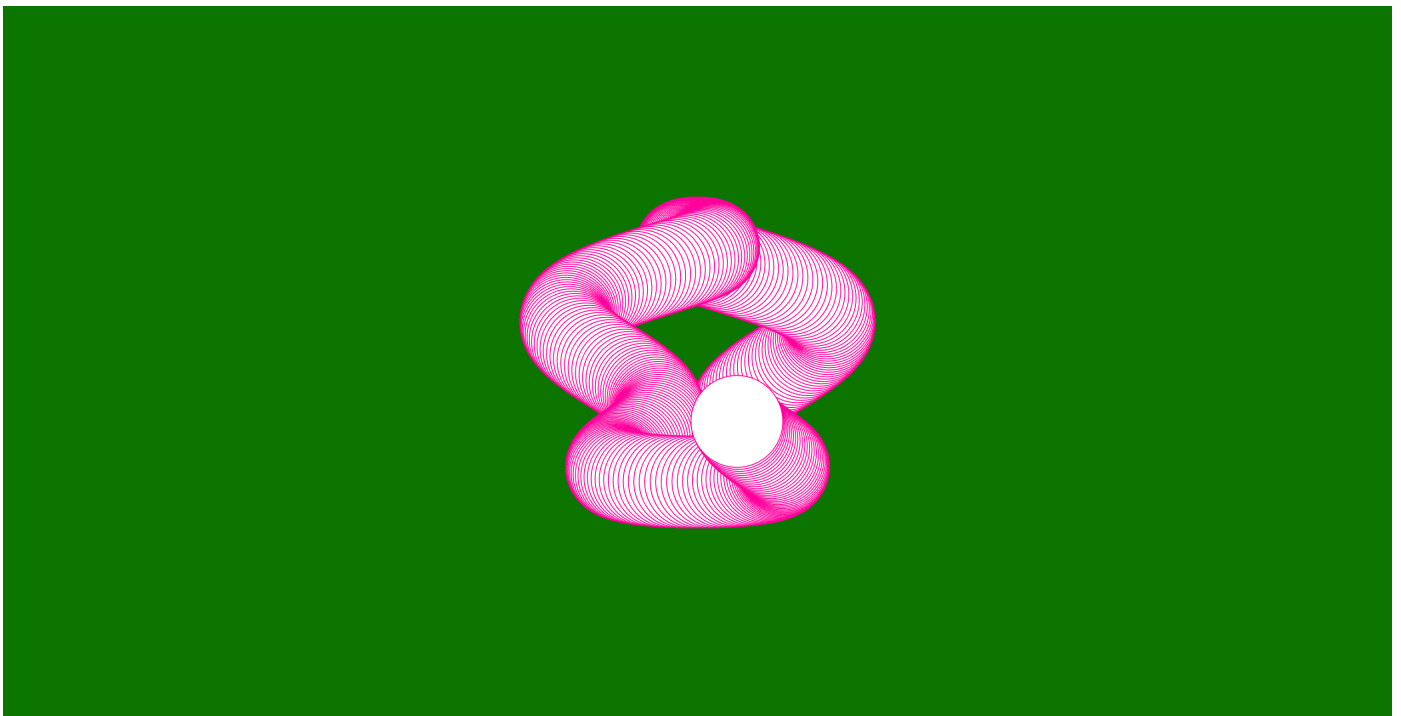
```

function setup() {
  createCanvas(windowWidth, windowHeight);
  stroke(255, 0, 157)
  background(12, 117, 0)
}

function draw() {
  // X-Bewegung > Kombi aus 2 Sinus-Wellen unterschiedliche Geschwindigkeit
  speedX = sin(frameCount * 0.02) * 100 + sin(frameCount * 0.08)*50
  // Y-Bewegung > Kombi aus 2 Cosinus-Wellen unterschiedliche Geschwindigkeit
  speedY = cos(frameCount * 0.02) * 100 + cos(frameCount * 0.02)*30

  // Zeichnet Kreis relativ zur Mitte des Bildschirms
  // Position durch speedX und speedY
  ellipse(width/2 + speedX,height/2 + speedY,100)
}

```



★ NOISE

```

function setup() {
  createCanvas(windowWidth, windowHeight);
  background(228, 255, 138)
  noFill()
}

function draw() {
  // Linie 1
  stroke(124, 138, 78)
  let x1 = noise(frameCount * 0.004) * width
  let y1 = noise(frameCount * 0.006) * height
  ellipse(x1, y1, 15)

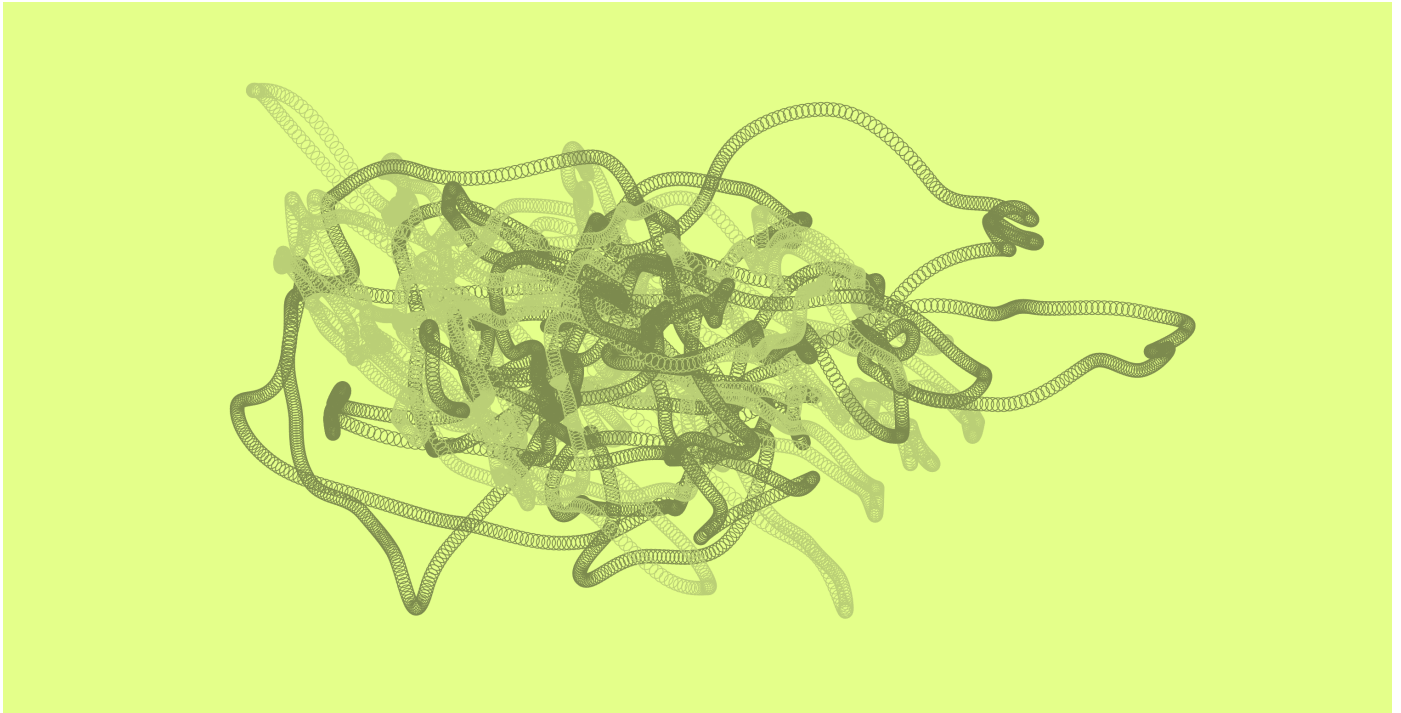
  // Linie 2
  stroke(186, 207, 118)

```

```

let x2 = noise(frameCount * 0.004 + 1000) * width
let y2 = noise(frameCount * 0.008 + 2000) * height
ellipse(x2, y2, 15)
}

```



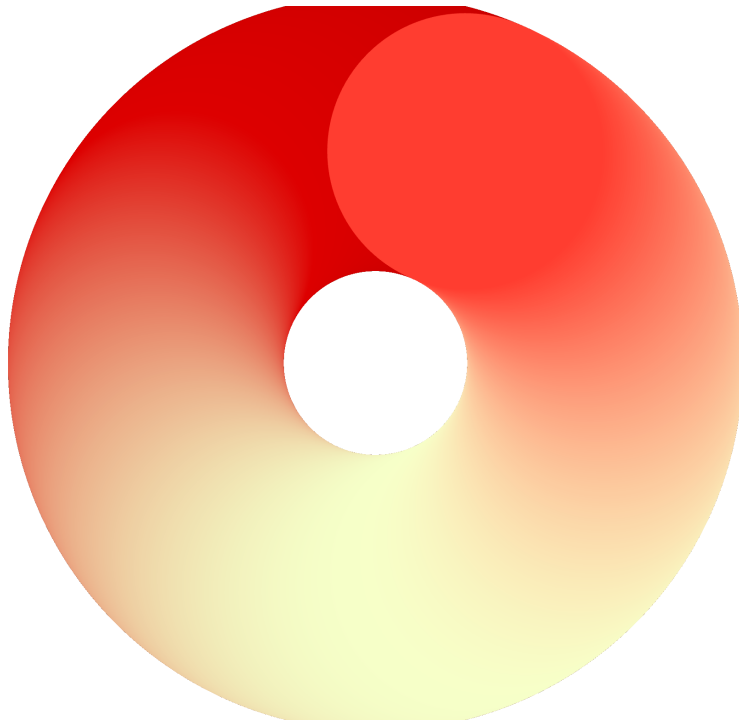
★ DYNAMISCHE FÜLLFARBE

```

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(255, 255, 255)
}

function draw() {
  // Dynamische Füllfarbe > RGB-Werte werden über Sinus gesteuert = weiche Farbverläufe
  fill(sin(frameCount*0.001)*255,sin(frameCount*0.006)*255,sin(frameCount*0.006)*200)
  noStroke()
  // zeichnet Ellipse > bewegt kreisförmig um Mitte
  ellipse(width/2 + sin(frameCount*0.01)*250, height/2 + cos(frameCount*0.01)*250, 300)
}

```



{Mittwoch} 06.05.2026

Vormittag

- Allgemeine Vorstellung von Andrea Zaccuri und seiner Arbeit
- Einführung in Hydra
 - Verstehen der Funktion auf der Github-Seite von Andrea
 - Experimente der Funktionen direkt auf der Github-Seite
<https://learn-andreazaccuri.azeta.workers.dev/260506-colabor/sources/>

Nachmittag

- Audioreaktives Verhalten mit P5Live und Hydra
- individuelle Zeit für Experimente

☆ HYDRA AUDIOREACTIVE

```
let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',  
'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']  
  
// ->Hydra in sandbox  
// sandbox - start  
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy  
  
s0.initP5() // send p5 to hydra = init -> camera  
P5.toggle(0) // optionally hide p5
```

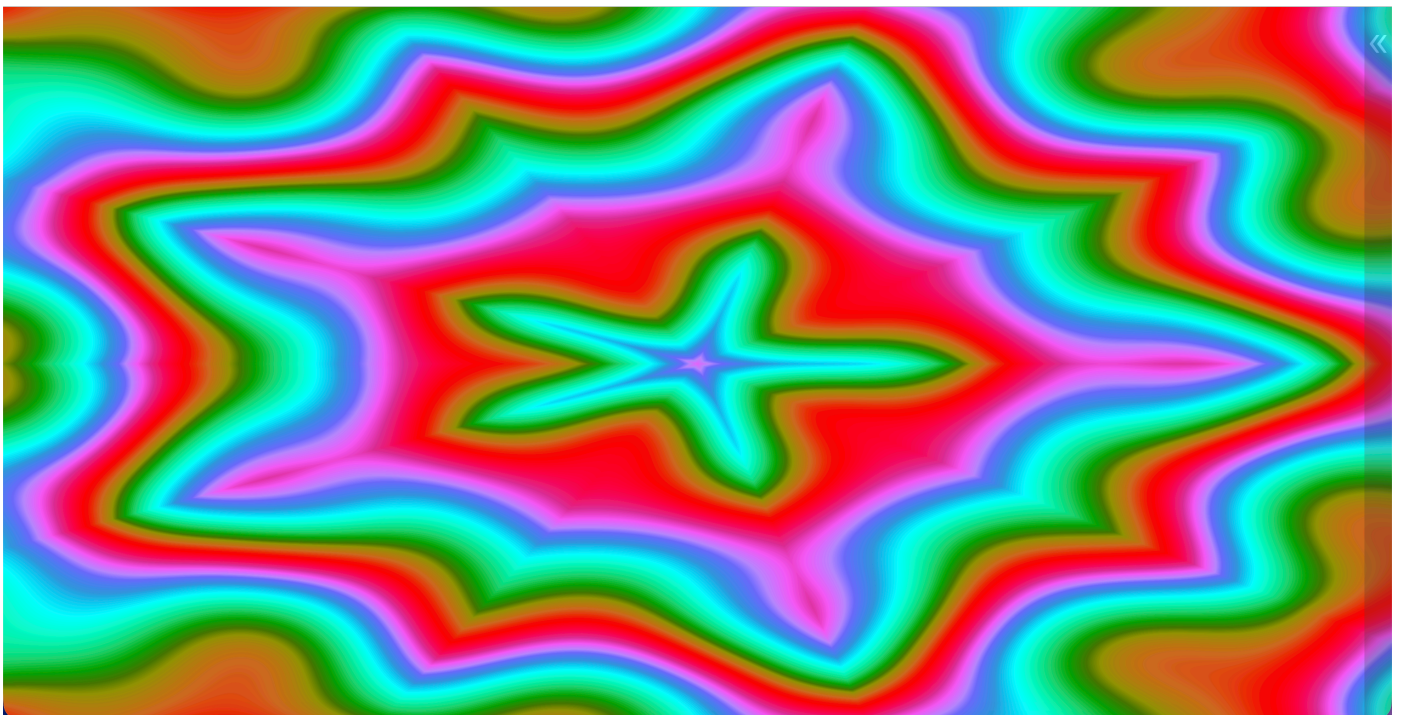
```

// erstellt Oszillator (Striche):
// Frequenz(Anz) / Bewegungsgeschwindigkeit / Farbraumverschiebung
osc(40, 0.01, 1.5)
// skaliert Bild abhängig von Bass > mehr Bass = grösseres Bild
.scale(() => 1 + a.fft[0] * 0.5)
// verzerrt Signal mit Noise(wolkig) > abhängig von Mids
.modulate(noise(3), () => a.fft[1] * 0.3)
// Kaleidoskop-Effekt > Anz der Segmente reagiert auf High-Mids (
.kaleid(() => 3 + a.fft[2] * 5)
// // Farbverschiebung von Höhen (Treble)
.hue(() => a.fft[3] * 5)
.out()

console.log(() => 3 + a.fft[2]*5)
// sandbox - end

// normal P5LIVE
function setup() {
  createCanvas(windowWidth, windowHeight)
}

```



☆ EXPERIMENT 1 HYDRA

```

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',
'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// ->Hydra in sandbox
// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra = init -> camera
P5.toggle(0) // optionally hide p5

```

```

osc(40, 0.01, 5)
.scale(() => 1 + a.fft[0] * 0.5) // bass
.modulate(noise(3), () => a.fft[1] * 0.3) // mids
.kaleid(() => 3 + a.fft[2] * 5) // hight mids
.hue(() => a.ff[3] * 5) // treble
.pixelate(100)
.out()

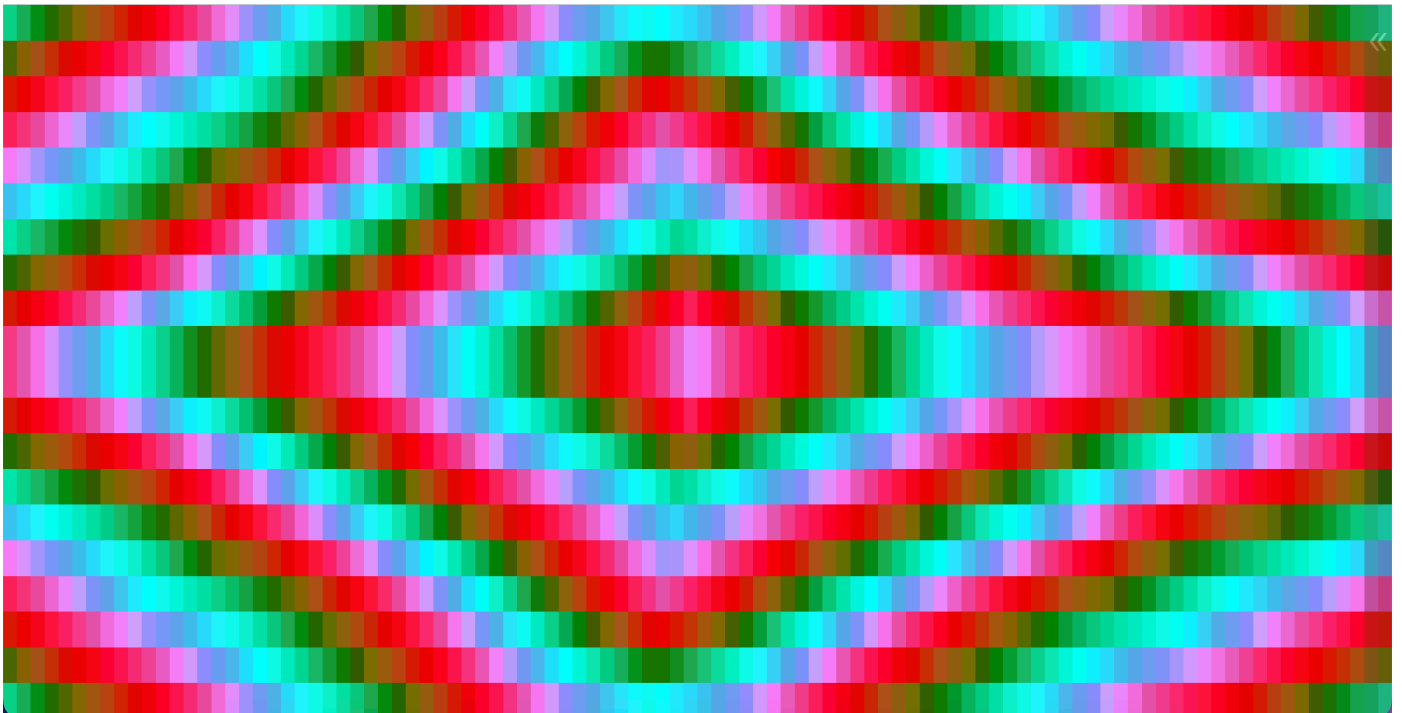
console.log(() => 3 + a.fft[2]*5)
// sandbox - end

// normal P5LIVE
function setup() {
  createCanvas(windowWidth, windowHeight)
}

// function draw() {
//   // clear()
//   circle(mouseX, mouseY, 100)}

// Wenn Shift + s auf Tastatur gedrückt wird > Screenshot
function keyPressed(){
  if(key == 'SS') {
    save('drawing.png')
  }
}

```



☆ EXPERIMENT 2 HYDRA

```

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',
'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// ->Hydra in sandbox
// sandbox - start

```

```

H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra = init -> camera
P5.toggle(0) // optionally hide p5

osc(10, 0.1, () => mouse.x * 0.1)
  .modulate(noise(3))
  .sub(gradient(),6)
  .mask(shape(1))
  .out(o0)
// sandbox - end

// normal P5LIVE
function setup() {
  createCanvas(windowWidth, windowHeight)
}
// function draw() {
//   // clear()
//   circle(mouseX, mouseY, 100)}

// Wenn Shift + s auf Tastatur gedrückt wird > Screenshot
function keyPressed(){
  if(key == 'SS') {
    save('drawing.png')
  }
}

```

☆ EXPERIMENTE 3 HYDRA

```

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',
'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// ->Hydra in sandbox
// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra = init -> camera
P5.toggle(0) // optionally hide p5

shape(200, 0.5, 1.5)
  .scale(0.5, 0.5)
  .color([0.5, 2].smooth(1), 0.3, 0)
  .repeat(2,2)
  .modulateScale(osc(3,0.5), -0.6)
  .add(o0, 0.5)
  .scale(0.9)
  .out()

```

```
// sandbox - end

// normal P5LIVE
function setup() {
  createCanvas(windowWidth, windowHeight)
}

// function draw() {
//   // clear()
//   circle(mouseX, mouseY, 100)}

// Wenn Shift + s auf Tastatur gedrückt wird > Screenshot
function keyPressed(){
  if(key == 'SS') {
    save('drawing.png')
  }
}
```

{Donnerstag} 07.05.2026

Vormittag

- Allgemeine Vorstellung von Jasmin Meerhoff und ihrer Arbeit
- Einführung in die «concrete poetry»

Nachmittag

- Erlernen verschiedener Funktionen mittels Text

Um ein Text-Slash zu setzen, muss zuerst ein Back-Slash geschrieben werden.

```
text("\\/", 100, 100)
```

★ TEXT WIEDERHOLUNG

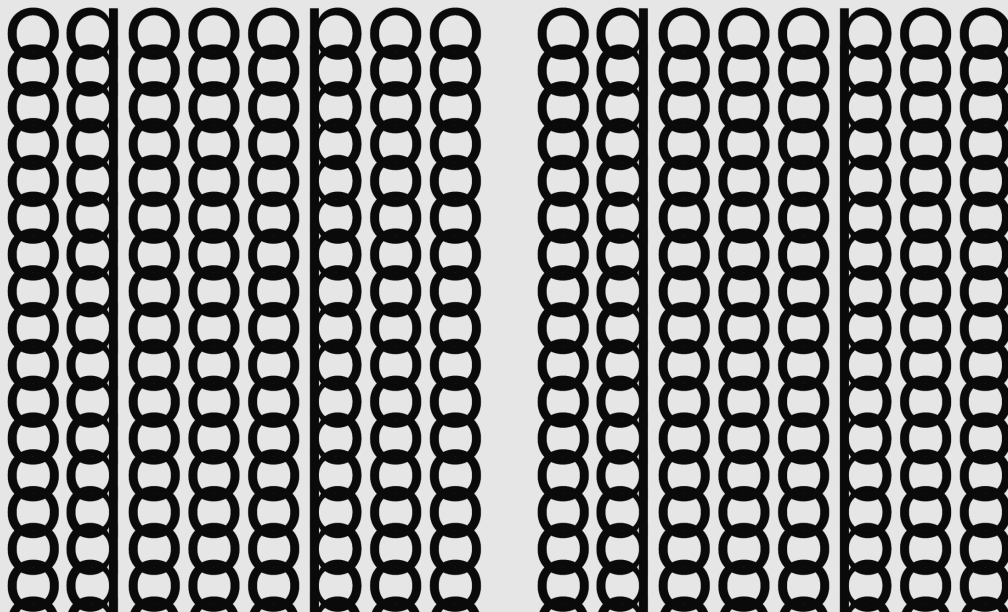
```
function setup() {
  createCanvas(windowWidth, windowHeight)
}
```



```
function draw() {
  let live = frameCount%10
  frameRate(5)

  background(230)
  fill(10)
  textSize(100)
  textFont("Parkinsans")
  // Schriftschnitte von System
  textStyle(NORMAL)
  // WORD => Umbruch nach Wörtern / CHAR => Umbruch nach einzelnen Buchstaben
  textWrap(WORD)
  // Ausrichtung => LEFT / CENTER / RIGHT
  textAlign(LEFT)
  // LineHeight => Ausgang gleiche Grösse wie Schrift
  textLeading(40)
  // repeat wiederholt den text

  text("oaooopoo ".repeat(100),
    100, 100, windowWidth/1.2, windowHeight)
}
```



★ TEXT ANIMATION

```
let live

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  live = frameCount%10
  frameRate(5)
```

```

background(230)
fill(10)
textSize(100)
textFont("monospace")
// Schriftschnitte von System
textStyle(ITALIC)
// WORD => Umbruch nach Wörtern / CHAR => Umbruch nach einzelnen Buchstaben
textWrap(CHAR)
// Ausrichtung => LEFT / CENTER / RIGHT
textAlign(LEFT)
// LineHeight => Ausgang gleiche Grösse wie Schrift
textLeading(32*(live/4))
// repeat wiederholt den text

text("aao aooo oaaa".repeat(100),
  10, 10, windowWidth/1.1, windowHeight)

// text("oaooopoo ".repeat(100),
// 100, 100, windowWidth/1.2, windowHeight)

// text("see this is a word that is breaking no matter what just break everything",
// 100, 100, windowWidth/1.2, windowHeight)
}

```

★ TEXT ERSETZEN 1

```

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  let live = frameCount%10
  let sine = floor(5*sin(frameCount/10)+5)
  let words = ["ouo", "----", "<3 <3", "hi", "$e y@", "%%%", "*****", "...", "huhu"]
  let rand = random(words)

  frameRate(5)

  background(0, 8, 250)
  fill(255, 255, 255)
  textSize(100)
  textFont("Parkinsans")
  textStyle(NORMAL)

```

```
textWrap(CHAR)
textAlign(LEFT)
textLeading(100)
text(words[0].replace(/o/g, "a"), 400, 400,
  windowWidth/1.1, windowHeight)
}
```



dua

★ TEXT ERSETZEN 2

```
function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  let live = frameCount%10
  let sine = floor(5*sin(frameCount/10)+5)
  let words = ["this is a longer sentence"]
  let rand = random(words)

  frameRate(5)

  background(0, 8, 250)
  fill(255, 255, 255)
  textSize(100)
  textFont("Parkinsans")
  textStyle(NORMAL)
  textWrap(CHAR)
  textAlign(LEFT)
  textLeading(100)
  text(words[0].replace(/g/, "-"), 400, 400,
    windowWidth/1.1, windowHeight)
}
```

this is a lon-er sentence

★ TEXT ERSETZEN 3

```
function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  let live = frameCount%10
  let sine = floor(5*sin(frameCount/10)+5)
  // müssen 10 Inhalte sein
  let words = ["xux", "ouo", "iii", "ala", "mhm", "lala", "hii", "<3", "383", "555" ]
  let rand = random(words)

  frameRate(5)

  background(0, 8, 250)
  fill(255, 255, 255)
  textSize(100)
  textFont("Parkinsans")
  textStyle(NORMAL)
  textWrap(CHAR)
  textAlign(LEFT)
  textLeading(100)
  text(words[live].replace(/[aieou]/g, "*-*-*").repeat(100), 50, 50,
    windowWidth/1.1, windowHeight)
}
```

★ EXPERIMENT

```
function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  let live = frameCount%10
  // let words = ["rain", "drops", "falling", "on", "my", "computer", "every", "day", "!"]
  // floor = runden
  let sine = floor(5*sin(frameCount/10)+5)
  let words = [ "<<>>", "----", "<3 <3", "hi", "$e y@", "%%%", "****", "...", "huhu", "n|ce" ]
  let rand = random(words)
  let bgColor = color(random(255), random(255), random(255))

  frameRate(3)

  background(bgColor)
  fill(255, 255, 255)
  textSize(100)
  textFont(random(["Parkinsans", "Circular Std", "Advine Pixel Demo"]))
  textStyle(random([NORMAL, ITALIC]))
  textWrap(CHAR)
  textAlign(LEFT)
  textLeading(100*(sine/10))
  text(words[sine].repeat(2000), 20, 20,
    windowWidth/1.1, windowHeight)
}
```

★ for-loop

```
function setup() {
  createCanvas(windowWidth, windowHeight)
```

```
frameRate(4)
noSmooth()
}

function draw() {
  let comma = "<<@>> "
  let space = ["-"]
  let commaline
  let count = frameCount%10

  background(0, 8, 250)
  fill(255, 255, 255)
  textSize(windowWidth/50)
  textFont("monospace")
  textWrap(CHAR)

  for (let i=0; i<11; i++) {
    space.push("**$**")
    comma = comma + space[i]
    commaline = comma.repeat(count+1+33)
    text(commaline, 100, commaline.length*i*count, windowWidth/count, windowHeight)
  }
}
```

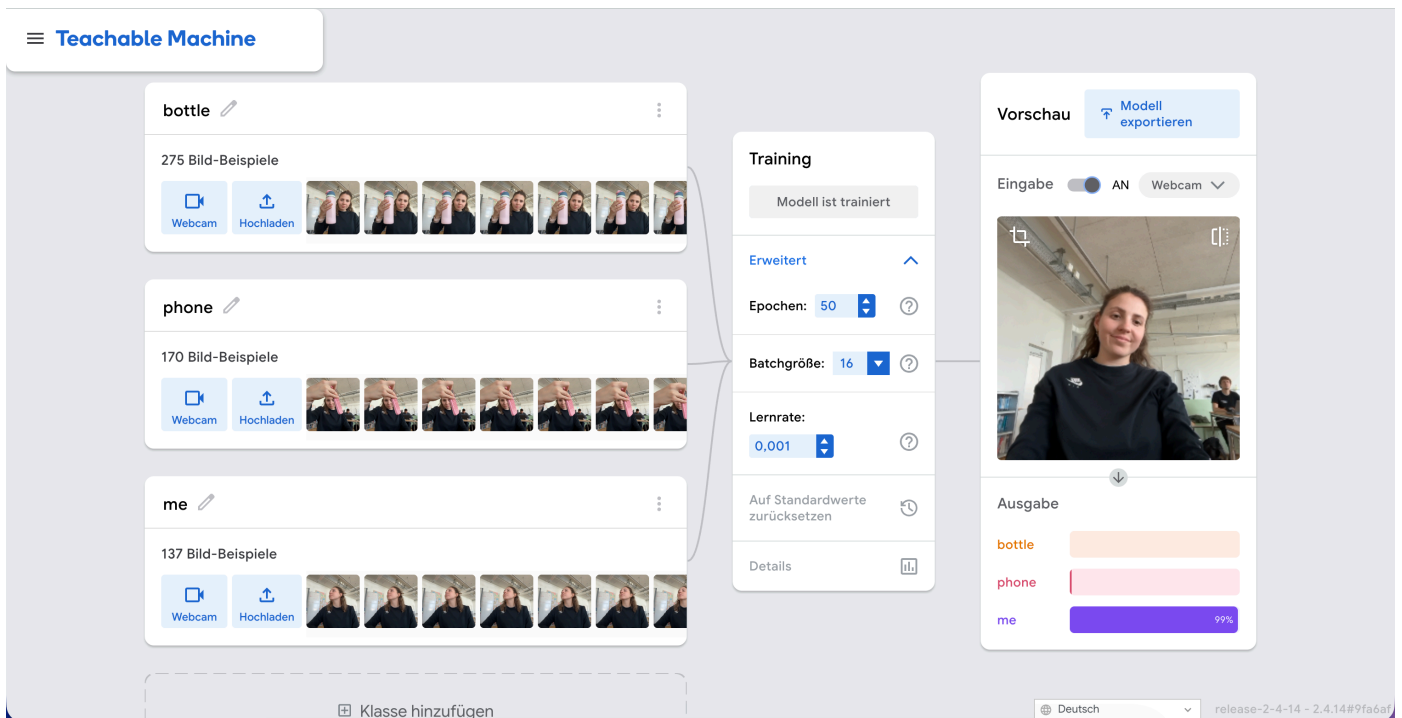
{Freitag} 08.05.2026

Vormittag

- Allgemeine Vorstellung von Paulina Zybinska und ihrer Arbeit
- Einführung in ml5.js
- Theorie zu «Machine Learning Models»

Nachmittag

- Verschiedene Experimente mit ml5.js in P5LIVE



☆ TECHABLE MACHINE IN P5LIVE

```
let libs = ["https://unpkg.com/ml5@1/dist/ml5.min.js", 'https://unpkg.com/hydra-synth',
'includes/libs/hydra-synth.js', 'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js',
'includes/libs/hy5.js']
let strength = 0;

let fx1 = 0
let fx2 = 0
let fx3 = 0

// sandbox - start
H.pixelDensity(2)
s0.initP5()
P5.toggle(0)

src(s0)
  .modulate(noize(10000), () => fx1 * 0.1)
  .modulateScale(osc(20), () => fx2)
  .modulate(voronoi(20, 2).luma(0.8).scale(0.99), () => fx3)
  // .add(src(s0).luma(0.9), () => fx3)
  // .add(src(s0).luma(() => 0.8 * a.fft[0]))
.out()
// sandbox - end

//let modellink -> von teachable machine link"
let modellink ="https://teachablemachine.withgoogle.com/models/QNMnjDWh6/"

// A variable to initialize the Image Classifier
let classifier;

// A variable to hold the video we want to classify
let video;
```

```

// Variable for displaying the results on the canvas
let label = "Model loading...";

function preload() {
  classifier = ml5.imageClassifier(modelLink);
}

function setup() {
  createCanvas(windowWidth, windowHeight);

  // Using webcam feed as video input, hiding html element to avoid duplicate with canvas
  video = createCapture(VIDEO, {flipped: true});
  video.size(width, height);
  video.hide();
  classifier.classifyStart(video, gotResult);
}

function draw() {
  // Each video frame is painted on the canvas
  image(video, 0, 0);
  //background(0, 0, 0)

  // Printing class with the highest probability on the canvas
  fill(255);
  textSize(32);
  text(label, width/2, height/2);

  if (label == "me"){
    circle(width/2, height/2, 100)
    noStroke()
    strength = 0.01
    fx1 = 1
    fx2 = 0
    fx3 = 0
  }
  else if (label == "phone"){
    rect(width/2, height/2, 200)
    noStroke()
    strength = 0.1
    fx1 = 0
    fx2 = 1
    fx3 = 0
  }
  else {
    triangle(width/2, height/2, width/2+200, height/2-200, width/2 + 400, height/2)
    noStroke()
    strength = 0.5

    fx1 = 0
    fx2 = 0
    fx3 = 1
  }
}

// Callback function for when classification has finished
function gotResult(results) {
  // Update label variable which is displayed on the canvas
  label = results[0].label;
}

```



```
}
```

☆ HAND POSE – PUNKT VERÄNDERN

```
let libs = ['https://unpkg.com/ml5@1/dist/ml5.min.js'];

let video;
let state = 'starting...';
let handPose;
let hands = [];
let options = { maxHands: 2, flipHorizontal: true };
let pScale = 20

let distTip = 0 // define variable for distance between finger tips
let distTip2 = 0

function preload() {
  handPose = ml5.handPose(options);
}

function setup() {
  createCanvas(windowWidth, windowHeight);

  video = createCapture(VIDEO, { flipped: true });
  video.size(width, height);
  video.hide();

  handPose.detectStart(video, gotHands);
  state = 'detecting hands';
}

function draw() {
  background(20, 100);
  //image(video,0,0) //show webcam

  for(let i = 0; i < hands.length; i++) {
    const hand = hands[i];
    for(let j = 0; j < hand.keypoints.length; j++) {
      const kp = hand.keypoints[j];
      const x = kp.x;
      const y = kp.y;
      fill(map(j, 0, hand.keypoints.length, 0, 255), 255, map(j, 0, hand.keypoints.length, 255, 0));
      noStroke();
      circle(x, y, pScale * sin(frameCount * 0.1 + j) % pScale);
    }
  }
}
```

```

noStroke();
fill(255);
text(state, 10, height - 10);

//according to ml5 handpose mode, thumb tip = keypoint 4,
//index finger tip = keypoint 8
//hands [0] = one of the hands; we have two = hands [0] and hands [1]

if(hands.length != 0) {

  stroke(255)
  strokeWeight(5)

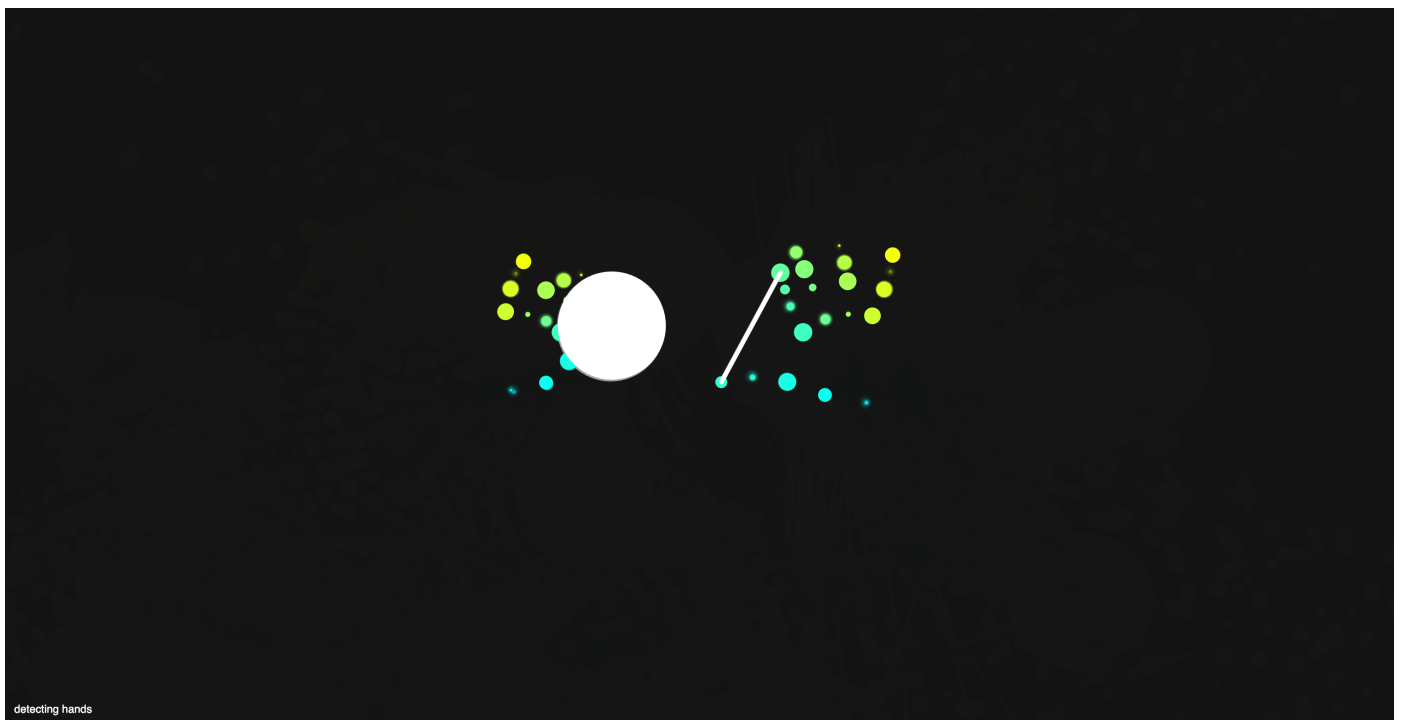
  distTip = dist(hands[0].keypoints[4].x, hands[0].keypoints[4].y,
    hands[0].keypoints[8].x, hands[0].keypoints[8].y)
  // if it detects a second hand
  if(hands[1]) {
    distTip2 = dist(hands[1].keypoints[4].x, hands[1].keypoints[4].y,
      hands[1].keypoints[8].x, hands[1].keypoints[8].y)
    line(hands[1].keypoints[4].x, hands[1].keypoints[4].y, hands[1].keypoints[8].x,
hands[1].keypoints[8].y)
  }

  line(hands[0].keypoints[4].x, hands[0].keypoints[4].y, hands[0].keypoints[8].x,
hands[0].keypoints[8].y)
  console.log(distTip)

  // if distance of tips of first detected hand is smaller than 30,
  // a circle is drawn
  // the distance of tips of second detected hand controls the size of the circle
  if(distTip < 30) {
    noStroke()
    circle(hands[0].keypoints[8].x, hands[0].keypoints[8].y, 50 + distTip2 * 0.5)
  }
}

function gotHands(results) {
  hands = results;
}

```



☆ HAND POSE – ZEICHNEN & PINSELGRÖSSE VERÄNDERN

```
let libs = ['https://unpkg.com/ml5@1/dist/ml5.min.js'];

let video;
let state = 'starting...';
let handPose;
let hands = [];
let options = { maxHands: 2, flipHorizontal: true };
let pScale = 20

let distTip = 0 // define variable for distance between finger tips
let distTip2 = 0

function preload() {
  handPose = ml5.handPose(options);
  //background(20, 43, 4)
}

function setup() {
  createCanvas(windowWidth, windowHeight);

  video = createCapture(VIDEO, { flipped: true });
  video.size(width, height);
  video.hide();

  handPose.detectStart(video, gotHands);
  state = 'detecting hands';
}

function draw() {
  //background(20, 100);
  //image(video,0,0) //show webcam
```

```

for(let i = 0; i < hands.length; i++) {
  const hand = hands[i];
  for(let j = 0; j < hand.keypoints.length; j++) {
    const kp = hand.keypoints[j];
    const x = kp.x;
    const y = kp.y;
    fill(map(j, 0, hand.keypoints.length, 0, 255), 255, map(j, 0, hand.keypoints.length, 255, 0));
    noStroke();
    //circle(x, y, pScale * sin(frameCount * 0.1 + j) % pScale);
  }
}

noStroke();
fill(235, 255, 222);
text(state, 10, height - 10);

//according to ml5 handpose mode, thumb tip = keypoint 4,
//index finger tip = keypoint 8
//hands [0] = one of the hands; we have two = hands [0] and hands [1]

if(hands.length != 0) {

  stroke(255)
  strokeWeight(5)

  distTip = dist(hands[0].keypoints[4].x, hands[0].keypoints[4].y,
    hands[0].keypoints[8].x, hands[0].keypoints[8].y)
  // if it detects a second hand
  if(hands[1]) {
    distTip2 = dist(hands[1].keypoints[4].x, hands[1].keypoints[4].y,
      hands[1].keypoints[8].x, hands[1].keypoints[8].y)
    //line(hands[1].keypoints[4].x, hands[1].keypoints[4].y, hands[1].keypoints[8].x,
hands[1].keypoints[8].y)
  }

  //line(hands[0].keypoints[4].x, hands[0].keypoints[4].y, hands[0].keypoints[8].x,
hands[0].keypoints[8].y)
  console.log(distTip)

  // if distance of tips of first detected hand is smaller than 30,
  // a circle is drawn
  // the distance of tips of second detected hand controls the size of the circle
  if(distTip < 30) {
    noStroke()
    circle(hands[0].keypoints[8].x, hands[0].keypoints[8].y, 50 + distTip2 * 0.5)
  }
}
}

function gotHands(results) {
  hands = results;
}

```

